

Software Vulnerabilities

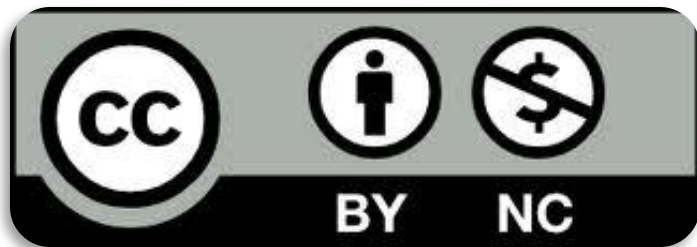


License & Disclaimer

2

License Information

This presentation is licensed under the
Creative Commons BY-NC License



To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc/3.0/legalcode>

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

Goal

3

- In this lecture we will introduce basic principles of Secure Programming.

Prerequisites

4

- Lecture:
 - Basic knowledge of C
 - Basic knowledge of Computer Programming

Outline

5

- Bugs and Vulnerabilities
- Categories of Code Vulnerabilities
- Source of Vulnerabilities
- Mitigations and Countermeasures

Software vulnerabilities...

6

- A *vulnerability* is a weakness which can be exploited by an *attacker* to perform *unauthorized actions* within your program
- To exploit a vulnerability, an attacker relies on **tools and techniques** related to a **software weakness**
- In this context, the vulnerability is also known as the **attack surface**

What is a *vulnerability*?

7

▣ **Question:** Is the following code *vulnerable*?

```
int authenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s",input);
    if (strcmp(password,input)==0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

What is a *vulnerability*?

8

▣ **Question:** Is the following code *vulnerable*?

Hardcoded password

```
int authenticate() {  
    char* password = "MyPassword!";  
    char* input = malloc(256);  
  
    printf("Enter the password: ");  
    scanf("%s",input);  
    if (strcmp(password,input)==0) {  
        printf("Authenticated!\n");  
        return 1;  
    } else {  
        printf("The password is wrong!\nPlease, try again!\n");  
        return 0;  
    }  
}
```


What is a *vulnerability*?

9

▣ **Question:** Is the following code *vulnerable*?

A buffer is allocated

```
int authenticate() {  
    char* password = "MyPassword!";  
    char* input = malloc(256);  
  
    printf("Enter the password: ");  
    scanf("%s",input);  
    if (strcmp(password,input)==0) {  
        printf("Authenticated!\n");  
        return 1;  
    } else {  
        printf("The password is wrong!\nPlease, try again!\n");  
        return 0;  
    }  
}
```

What is a *vulnerability*?

10

▣ **Question:** Is the following code *vulnerable*?

User input

```
int authenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s", input);
    if (strcmp(password, input) == 0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

What is a *vulnerability*?

11

▣ **Question:** Is the following code *vulnerable*?

```
int authenticate() {  
    char* password = "MyPassword!";  
    char* input = malloc(256);  
  
    printf("Enter the password: ");  
    scanf("%s", input);  
    if (strcmp(password, input) == 0) {  
        printf("Authenticated!\n");  
        return 1;  
    } else {  
        printf("The password is wrong!\nPlease, try again!\n");  
        return 0;  
    }  
}
```

String comparison

What is a *vulnerability*?

12

▣ **Question:** Is the following code *vulnerable*? **Yes!**

There are two vulnerabilities in this code:

- ▣ *Hardcoded password*
- ▣ *Potential buffer overflow*

```
int authenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s",input);
    if (strcmp(password,input)==0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

Vulnerabilities vs bugs...

13

- Vulnerabilities are more general than bugs.
- According to the IEEE Terminology¹
 - *Bug*: an *error* or a *fault* that causes a *failure*
 - *Error*: a human action that produces an incorrect result
 - *Fault*: an incorrect step, process, or data definition in a computer program
 - *Failure*: the inability of software to perform its required functions within specified performance requirements

¹IEEE Standard Glossary of Software Engineering Terminology

Code vulnerabilities...

14

- ▣ *Information leakage*
- ▣ *Buffer overflow*
- ▣ *Race condition*
- ▣ *Invalid data processing*

Code vulnerabilities...

15

- **Information leakage:** information is unintentionally disclosed to the end-user and used by attackers to breach application security

Code vulnerabilities...

16

- ❑ **Information leakage:** information is unintentionally disclosed to unauthorized users or attackers

```
int authenticate() {  
    char* password = "MyPassword!";  
    char* input = malloc(256);  
  
    printf("Enter the password: ");  
    scanf("%s", input);  
    if (strcmp(password, input) == 0) {  
        printf("Authenticated!\n");  
        return 1;  
    } else {  
        printf("The password is wrong!\nPlease, try again!\n");  
        return 0;  
    }  
}
```


Code vulnerabilities...

Vedi slide di
Approfondimento
“Memory Corruption”

17

- **Buffer overflow:** writing operations on data can overrun the buffer's boundary and overwrite adjacent memory locations. This may cause:
 - execution of malicious code
 - privileges escalation
- Common and very harmful vulnerability!

Code vulnerabilities...

18

- **Buffer overflow:** writing operations on data can overrun the buffer and corrupt memory

```
int authenticate() {  
    char* password = "MyPassword";  
    char* input = malloc(256);  
    printf("Enter the password: ");  
    scanf("%s", input);  
    if (strcmp(password, input) == 0) {  
        printf("Authenticated!\n");  
        return 1;  
    } else {  
        printf("The password is wrong!\nPlease, try again!\n");  
        return 0;  
    }  
}
```

Code vulnerabilities...

19

- ▣ **Race condition:** the small window of time between appliance of a security control and use of services in a system allows to change program behaviour...
 - ▣ It is the result of interferences among multiple threads running in the system and sharing the same resources

Code vulnerabilities...

20

- ❑ **Race condition:** the small window of time between applying a service in a system and the service is no longer available...
 - ❑ It is a race between the user and the system to reach the service.

```
int raceCondition() {
    char * fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;
    if(!access(fn, W_OK)){
        scanf("%50s", buffer );
        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    } else {
        printf("No permission \n");
    }
}
```

Code vulnerabilities...

21

- **Race condition:** the small window of time between applying a system call and the system's response...
□ It is a race between the user's request and the system's response...

```
int raceCondition() {  
    char * fn = "/tmp/XYZ";  
    char buffer[60];  
    FILE *fp;  
    if(!access(fn, W_OK)){  
        scanf("%50s", buffer);  
        fp = fopen(fn, "a+");  
        fwrite("\n", sizeof(char), 1, fp);  
        fwrite(buffer, sizeof(char), strlen(buffer), fp);  
        fclose(fp);  
    } else {  
        printf("No permission \n");  
    }  
}
```

Access rights are verified

Code vulnerabilities...

22

- **Race condition:** the small window of time between applying a system call and the system's response. It leads to unpredictable results.
- It is a race between the program and the system. For example, if a program is waiting for an input, file can be linked to another /etc/shadow file...

```
int raceCondition() {  
    char * fn = "/tmp/XYZ";  
    char buffer[60];  
    FILE *fp;  
    if(!access(fn, W_OK)){  
        scanf("%50s", buffer );  
        fp = fopen(fn, "a+");  
        fwrite("\n", sizeof(char), 1, fp);  
        fwrite(buffer, sizeof(char), strlen(buffer), fp);  
        fclose(fp);  
    } else {  
        printf("No permission \n");  
    }  
}
```

Code vulnerabilities...

23

- ❑ **Invalid data processing:** data are processed with wrong or partial assumptions that allow the attacker to enable unwanted behaviours or execute malicious code

Sources of vulnerabilities...

24

- ❑ Complexity, inadequacy, and (uncontrolled) changes
- ❑ Incorrect or changing assumptions (capabilities, inputs, outputs)
- ❑ Flawed specifications and designs
- ❑ Poor implementation of software interfaces (input validation, error and exception handling)
- ❑ Unintended, unexpected interactions with other components with the software's execution environment
- ❑ ***Inadequate knowledge of secure coding practices***

How to avoid vulnerabilities?

25

- The large part of vulnerabilities are consequence of *errors* or *bad choices* in software development
- The first (and main) *countermeasure* to avoid *software vulnerabilities* is to develop *good software*

Good practices

26

1. *The simpler, the better...* keep your code simple!
2. *All that glitters is not gold...* keep track of assumptions made at design time and check them at runtime
3. *The evil always come from details...* always test your code!
4. *If you think education is expensive, try ignorance...* spent time *to learn* and *to study* the right solution and technology for your software

Software Vulnerabilities



Static and Dynamic Analysis

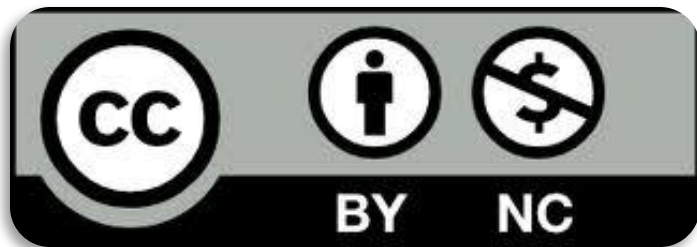


License & Disclaimer

29

License Information

This presentation is licensed under the
Creative Commons BY-NC License



To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc/3.0/legalcode>

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

Goal

30

- In this lecture we will study common techniques and tools that can be used to analyze statically and dynamically binary executables.

Prerequisites

31

- Lecture:
 - Basic knowledge of C Programming
 - ELF Binary formats
 - Debugging tools

Outline

32

- ▣ Static Analysis
 - ▣ Extracting data from binaries
 - ▣ Discovery of vulnerabilities
- ▣ Dynamic Analysis
 - ▣ Tools for dynamic analysis
 - ▣ Memory leaks
 - ▣ Anti-debug protection techniques

Static Analysis

33

- ▣ *Static program analysis* is the analysis of computer software that is performed without actually executing programs
 - ▣ in contrast with *dynamic analysis* which is analysis performed on programs while they are executing
- ▣ Static analysis can be performed on...
 - ▣ the source code
 - ▣ the object code
- ▣ In this module, we will focus on *static analysis* performed on the *object code*

Gathering info from binary files

34

- Several tools are available to extract information from an ELF file, such as:
 - `objdump`
 - `readelf`
 - `strings`
- These tools can be used to gather information from binaries that can be used to discover vulnerabilities

Gathering info from binary files

35

- Given a binary file we can:
 - check if the file is **executable** or not
 - discover the **architecture** for which the binary has been compiled
 - collect **symbols** and **strings** used in the program
 - check if there is a **running process** associated with the binary
 - read the SHA of a file and check if it is associated with some **malicious software**
 - identify function names and used **libraries**

Identifying vulnerabilities

36

- Information collected from binaries can be used to discover program vulnerabilities:
 - where a program takes an input and if this is validated
 - if memory is safely managed
 - if up-to-date libraries or third-party frameworks are used
 - how the application is compiled
 - if (static) strings are used to record sensitive data

Example

37

- Let us assume we have a program named `guessmyname`. When executed this program just ask the user to guess a name:

```
CC> ./guessmyname
Guess my name: Paulo
I am sorry, this is not my name!
Please, try again.
CC> 
```

- We can use `readelf` to check if the name to guess is *hardcoded* in the file

Example

38

- We can read the content of `.rodata` section:

```
CC> readelf -x .rodata guessmyname

Hex dump of section '.rodata':
0x00002000 01000200 00000000 43616c65 66004775 .....Calef.Gu
0x00002010 65737320 6d79206e 616d653a 20002531 ess my name: .%1
0x00002020 30730000 00000000 57656c6c 20646f6e 0s.....Well don
0x00002030 65212059 6f752068 61766520 67756573 e! You have gues
0x00002040 73656420 6d79206e 616d6521 00000000 sed my name!....
0x00002050 4920616d 20736f72 72792c20 74686973 I am sorry, this
0x00002060 20697320 6e6f7420 6d79206e 616d6521 is not my name!
0x00002070 00506c65 6173652c 20747279 20616761 .Please, try aga
0x00002080 696e2e00 in..

CC> █
```

Example

39

- We can now use the string we have discovered in our program:

```
CC> ./guessmyname  
Guess my name: Calef  
Well done! You have guessed my name!  
CC> █
```

WARNING: To avoid information leakage, never hardcode secrets in your program!

Example

40

- We can also use the tool **strings** to extract all the constant strings used in the code:

```
CC> strings guessmyname
/lib64/ld-linux-x86-64.so.2
libc.so.6
__isoc99_scanf
puts
printf
malloc
__cxa_finalize
strcmp
__libc_start_main
GLIBC_2.7
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
u+UH
[IA\A]A^A_
Calef
Guess my name:
%10s
Well done! You have guessed my name!
I am sorry, this is not my name!
Please, try again.
:*3$"
GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.8060
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
```


Example

41

- Looking at the code, we can observe that the name is in fact *hardcoded*:

```
int main(int argc, char** argv) {
    char* name = "Calef";
    char* input = malloc(10);

    printf("Guess my name: ");
    scanf("%10s",input);
    if (strcmp(name,input)==0) {
        printf("Well done! You have guessed my name!\n");
    } else {
        printf("I am sorry, this is not my name!\n");
        printf("Please, try again.\n");
    }
}
```

From static to dynamic analysis

42

- We have seen how information can be collected from a binary file
- We learnt that, to avoid information leakage, private data should not be hardcoded
- This is not enough!
- Program data can be inspected at runtime!

Dynamic analysis

43

- **Dynamic analysis** is a program analysis technique performed either on virtual or on real processor
- This kind of analysis is performed using tools that are able to **observe** program execution:
 - *dynamic memory analysis*
 - *dynamic disassembly*

GDB in action

44

- A new version of our *game* has been released where developers say they have fixed the previously identified vulnerabilities:

```
CC> ./guessmyname2
Guess my name: Calef
I am sorry, this is not my name!
Please, try again.
CC> █
```

- They also changed the name...

Example

45

- We read the content of **.rodata** section:

```
CC> readelf -x .rodata guessmyname2

Hex dump of section '.rodata':
0x00002000 01000200 00000000 47756573 73206d79 .....Guess my
0x00002010 206e616d 653a2000 25313073 00000000  name: .%10s....
0x00002020 57656c6c 20646f6e 65212059 6f752068 Well done! You h
0x00002030 61766520 67756573 73656420 6d79206e ave guessed my n
0x00002040 616d6521 00000000 4920616d 20736f72 ame!....I am sor
0x00002050 72792c20 74686973 20697320 6e6f7420 ry, this is not
0x00002060 6d79206e 616d6521 00506c65 6173652c my name!.Please,
0x00002070 20747279 20616761 696e2e00          try again..

CC> █
```

- The lesson has been learnt... no name is evident in the program
- We can use **gdb** to perform dynamic analysis of our code

Example

46

- We start gdb, load the binary, and set a breakpoint at function main:

```
GEF for linux ready, type `gef' to start, `gef config' to configure
92 commands loaded for GDB 9.2 using Python engine 3.8
gef> file guessmyname2
Reading symbols from guessmyname2...
(No debugging symbols found in guessmyname2)
gef> break main
Breakpoint 1 at 0x12b8
gef> run
```

Example

47

- We start gdb, load the binary, and set a breakpoint at function main:

```
GEF for linux ready, type `gef' to start, `gef config' to configure
92 commands loaded for GDB 9.2 using Python engine 3.8
→ gef> file guessmyname2
Reading symbols from guessmyname2...
(No debugging symbols found in guessmyname2)
gef> break main
Breakpoint 1 at 0x12b8
gef> run
```

- Note that we have not debug data in the object files

Example

48

- We start gdb, load the binary, and set a breakpoint at function main:

```
GEF for linux ready, type `gef' to start, `gef_config' to configure
92 commands loaded for GDB 9.2 using Python engine 3.8
gef> file guessmyname2
Reading symbols from guessmyname2...
(No debugging symbols found in guessmyname2)
→ gef> break main
Breakpoint 1 at 0x12b8
gef> run
```

- Note that we have not debug data in the object files

Example

49

- We start gdb, load the binary, and set a breakpoint at function main:

```
GEF for linux ready, type `gef' to start, `gef_config' to configure
92 commands loaded for GDB 9.2 using Python engine 3.8
gef> file guessmyname2
Reading symbols from guessmyname2...
(No debugging symbols found in guessmyname2)
gef> break main
Breakpoint 1 at 0x12b8
gef> run
```

- Note that we have not debug data in the object files

Example

50

```
[ Legend: Modified register | Code | Heap | Stack | String ]

registers
$eax : 0xf7fb6808 → 0xffffd13c → 0xffffd324 → "SHELL=/bin/bash"
$ebx : 0x0
$ecx : 0x6a309012
$edx : 0xffffd0c4 → 0x00000000
$esp : 0xffffd09c → 0xf7debee5 → <__libc_start_main+245> add esp, 0x10
$ebp : 0x0
$esi : 0xf7fb4000 → 0x001e6d6c
$edi : 0xf7fb4000 → 0x001e6d6c
$eip : 0x565562b8 → <main+0> endbr32
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x0023 $ss: 0x002b $ds: 0x002b $es: 0x002b $fs: 0x0000 $gs: 0x0063

stack
0xffffd09c +0x0000: 0xf7debee5 → <__libc_start_main+245> add esp, 0x10 ← $esp
0xffffd0a0 +0x0004: 0x00000001
0xffffd0a4 +0x0008: 0xffffd134 → 0xffffd2fb → "/home/loretti/CC/LECTURES/S1/guessmyname2"
0xffffd0a8 +0x000c: 0xffffd13c → 0xffffd324 → "SHELL=/bin/bash"
0xffffd0ac +0x0010: 0xffffd0c4 → 0x00000000
0xffffd0b0 +0x0014: 0xf7fb4000 → 0x001e6d6c
0xffffd0b4 +0x0018: 0xf7ff0000 → 0x0002bf24
0xffffd0b8 +0x001c: 0xffffd118 → 0xffffd2fb → "/home/loretti/CC/LECTURES/S1/guessmyname2"

code:x86:32
0x565562b3 <obfuscatedName+102> mov ebx, DWORD PTR [ebp-0x4]
0x565562b6 <obfuscatedName+105> leave
0x565562b7 <obfuscatedName+106> ret
→ 0x565562b8 <main+0> endbr32
0x565562bc <main+4> lea ecx, [esp+0x4]
0x565562c0 <main+8> and esp, 0xffffffff
0x565562c3 <main+11> push DWORD PTR [ecx-0x4]
0x565562c6 <main+14> push ebp
0x565562c7 <main+15> mov ebp, esp

threads
[#0] Id 1, Name: "guessmyname2", stopped 0x565562b8 in main (), reason: BREAKPOINT

trace
[#0] 0x565562b8 → main()

gef> ni
```

We can execute the program *step-by-step* by using command *nexti* (abbrv. *ni*)

Example

51

```
[ Legend: Modified register | Code | Heap | Stack | String ]

registers
$eax : 0x5655a1a0 → "Ulisse"
$ebx : 0x56558fc8 → 0x00003ed0
$ecx : 0x21e59
$edx : 0x5655a1a8 → 0x00000000
$esp : 0xffffd064 → 0x56558fc8 → 0x00003ed0
$ebp : 0xffffd088 → 0x00000000
$esi : 0xf7fb4000 → 0x001e6d6c
$edi : 0xf7fb4000 → 0x001e6d6c
$eip : 0x565562e4 → <main+44> push 0xa
$eflags: [Zero carry parity ADJUST SIGN trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x0023 $ss: 0x002b $ds: 0x002b $es: 0x002b $fs: 0x0000 $gs: 0x0063

stack
0xffffd064 +0x0000: 0x56558fc8 → 0x00003ed0 ← $esp
0xffffd068 +0x0004: 0xffffd088 → 0x00000000
0xffffd06c +0x0008: 0x565562de → <main+38> mov DWORD PTR [ebp-0x10], eax
0xffffd070 +0x000c: 0x00000001
0xffffd074 +0x0010: 0xffffd13a → 0xffffd2fb → "/home/Loretti/CC/LECTURES/S1/guessmyname2"
0xffffd078 +0x0014: 0x5655a1a0 → "Ulisse"
0xffffd07c +0x0018: 0x565563a1 → <main+33> lea ebx, [ebp-0xfc]
0xffffd080 +0x001c: 0xffffd0a0 → 0x00000001

code:x86:32
0x565562d9 <main+33> call 0x5655624d <obfuscatedName>
0x565562de <main+38> mov DWORD PTR [ebp-0x10], eax
0x565562e1 <main+41> sub esp, 0xc
→ 0x565562e4 <main+44> push 0xa
0x565562e6 <main+46> call 0x565560d0 <malloc@plt>
0x565562eb <main+51> add esp, 0x10
0x565562ee <main+54> mov DWORD PTR [ebp-0xc], eax
0x565562f1 <main+57> sub esp, 0xc
0x565562f4 <main+60> lea eax, [ebx-0x1fc0]

threads
[#0] Id 1, Name: "guessmyname2", stopped 0x565562e4 in main (), reason: SINGLE STEP

trace
[#0] 0x565562e4 → main()

gef> |
```

After some steps a potential name appears in the stack!

Example

52

- We can try with the discovered name:

```
CC> ./guessmyname2  
Guess my name: Ulisse  
Well done! You have guessed my name!  
CC> █
```

Example

53

- If we examine the code we can observe that in this case the password is obfuscated, however it is revealed when saved in a variable:

```
int main(int argc, char** argv) {  
    char* name = obfuscatedName();  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s",input);  
    if (strcmp(name,input)==0) {  
        printf("Well done! You have guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\n");  
        printf("Please, try again.\n");  
    }  
}
```

Example

54

- The same information can be extracted by using the tool **ltrace** that allows to trace calls to libraries:

```
CC> ltrace ./guessmyname2
__libc_start_main(0x4006cb, 1, 0x7ffc04c1b0f8, 0x400750 <unfinished ...>
malloc(7) = 0x25a9010
malloc(10) = 0x25a9030
printf("Guess my name: ") = 15
__isoc99_scanf(0x4007e8, 0x25a9030, 0x7f3b635e9780, 15Guess my name: Harry
) = 1
strcmp("Ulisse", "Harry") = 13
puts("I am sorry, this is not my name!...I am sorry, this is not my name!
Please, try again!
) = 52
+++ exited (status 0) +++
CC> █
```

How can we patch this program?

55

Versione
non
vulnerabile
a questo
tipo di
attacco

```
int main() {  
    char* name = readEncryptedName();  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s",input);  
    if (strcmp(name,encrypt(input))==0) {  
        printf("Well done! You guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\nPlease, try again!\n");  
    }  
    return 0;  
}
```

How can we patch this program?

56

Versione
non
vulnerabile
a questo
tipo di
attacco

```
int main() {  
    char* name = readEncryptedName();  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s",input);  
    if (strcmp(name,encrypt(input))==0) {  
        printf("Well done! You guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\nPlease, try again!\n");  
    }  
    return 0;  
}
```

Read encrypted name from a file

How can we patch this program?

57

Versione
non
vulnerabile
a questo
tipo di
attacco

```
int main() {  
    char* name = readEncryptedName();  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s", input);  
    if (strcmp(name, encrypt(input)) == 0) {  
        printf("Well done! You guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\nPlease, try again!\n");  
    }  
    return 0;  
}
```

Encrypt the name read from input and compare it with the one we have!

How can we patch this program?

58

Versione
non
vulnerabile
a questo
tipo di
attacco

```
int main() {  
    char* name = readEncryptedName();  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s",input);  
    if (strcmp(name,encrypt(input))==0) {  
        printf("Well done! You guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\nPlease, try again!\n");  
    }  
    return 0;  
}
```

The implementation of 'readEncryptedName' and 'encrypt' is left as an exercise!

Anti-debugging techniques

59

- ❑ One can limit *dynamic analysis* by using *anti-debugging techniques*
- ❑ The idea is to execute specific code that causes errors when a debugger tries to *trace* a program
- ❑ One of the simpler approach is the one based on *self-debugging*

Self-Debugging

60

- A *new process* is created to *trace* execution of our program (like a debugger)
 - Function *ptrace* is used
- If a debugger is running an error is generated and the program terminates
- In *C self-debugging* is implemented by relying on function *ptrace* available in *sys/ptrace.h*

Self-Debugging: Example

61

```
int main(int argc, char** argv) {
    if (ptrace(PTRACE_TRACEME,0,1,0) == -1) {
        return -1;
    }
    char* name = "Calef";
    char* input = malloc(10);

    printf("Guess my name: ");
    scanf("%10s",input);
    if (strcmp(name,input)==0) {
        printf("Well done! You have guessed my name!\n");
    } else {
        printf("I am sorry, this is not my name!\n");
        printf("Please, try again.\n");
    }
}
```

Self-Debugging: Example

62

```
int main(int argc, char** argv) {  
    if (ptrace(PTRACE_TRACEME, 0, 1, 0) == -1) {  
        return -1;  
    }  
    char* name = "Cater";  
    char* input = malloc(10);  
  
    printf("Guess my name: ");  
    scanf("%10s", input);  
    if (strcmp(name, input) == 0) {  
        printf("Well done! You have guessed my name!\n");  
    } else {  
        printf("I am sorry, this is not my name!\n");  
        printf("Please, try again.\n");  
    }  
}
```

Bypassing Self-Debugging

63

- We can still use a debugger to bypass *self-debugging*
- We have only to:
 - Find where *ptrace* is invoked
 - Let the debugger *skip* this invocation
- There are *reverse engineering* techniques that can help in this task

Disassembly at work: An Example

64

- Let us consider the program *findmysecret*:

```
CC> ./findmysecret  
You will never know my secret!  
CC>
```

- We know that a secret is stored somewhere in the program and we must discover it

Disassembly at work: An Example

65

- ❑ After using a tool like Ghidra, we can use *gdb* to call one of the program's functions
- ❑ First, we must load our program and set a breakpoint in the main:

```
gef> file findmysecret
Reading symbols from findmysecret...
(No debugging symbols found in findmysecret)
gef> break main
Breakpoint 1 at 0x1234
gef> run
```

- ❑ After that we can call the function `functionOne()` and see the secret:

```
trace
[#0] 0x56556234 → main()

gef> call (void *) functionOne()
My secret is 42!

$2 = (void *) 0x12
gef>
```

Static and Dynamic Analysis





Esercizio Guidato con Ghidra

Giacomo Bettini

giacomo.bettini@unife.it

Esercizio guidato

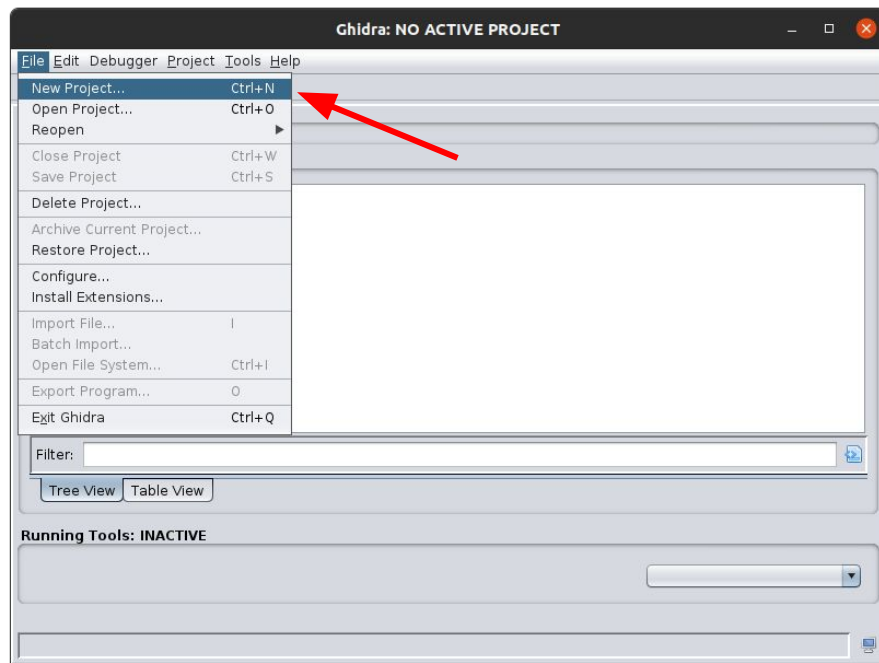
- Viene fornito un file eseguibile chiamato “esempio”, che richiede una password che non conosciamo:

PROBLEMI OUTPUT CONSOLE DI DEBUG TERMINALE

```
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$ ./esempio
Usage: ./esempio password
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$ ./esempio test_password
Password wrong...
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$
```

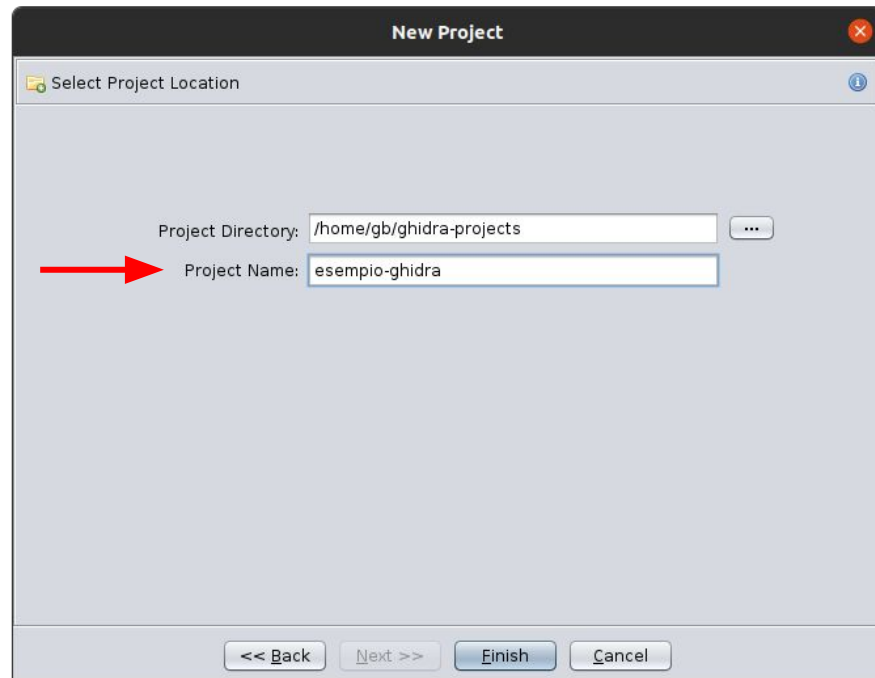
Esercizio guidato

- Iniziamo aprendo Ghidra e creando un nuovo progetto



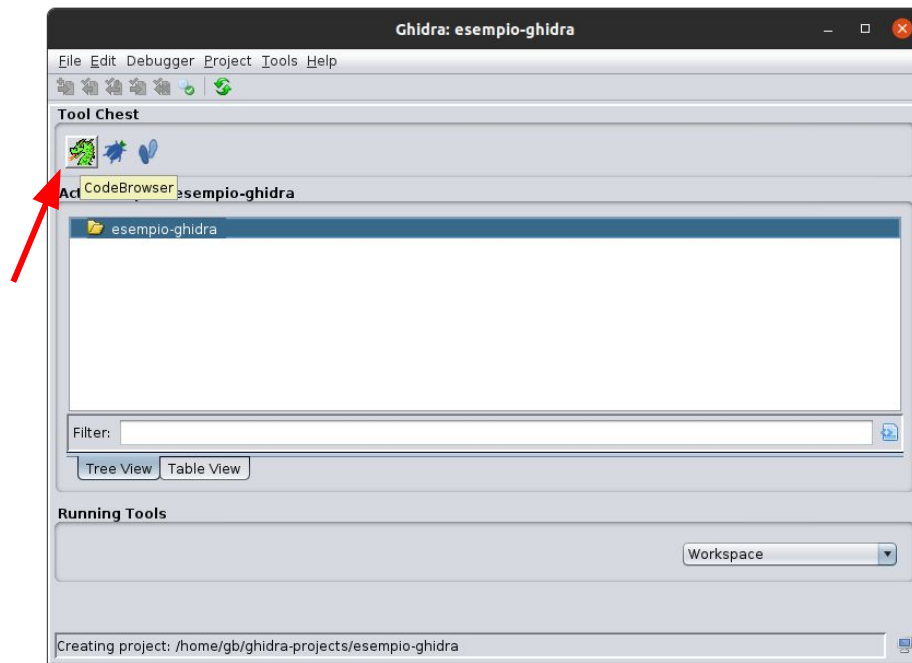
Esercizio guidato

- Diamo un nome al progetto



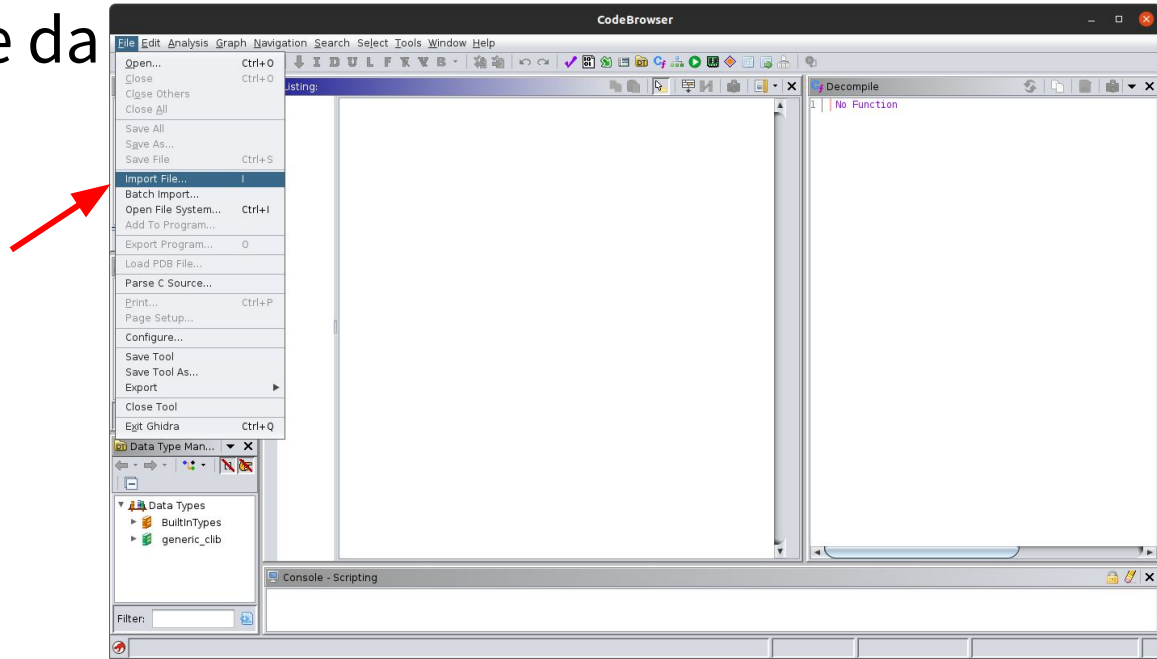
Esercizio guidato

- Ora apriamo il CodeBrowser



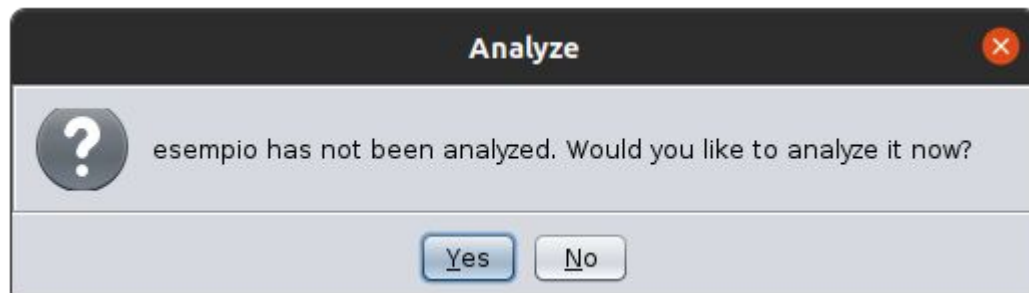
Esercizio guidato

- Importiamo il file da disassemblare



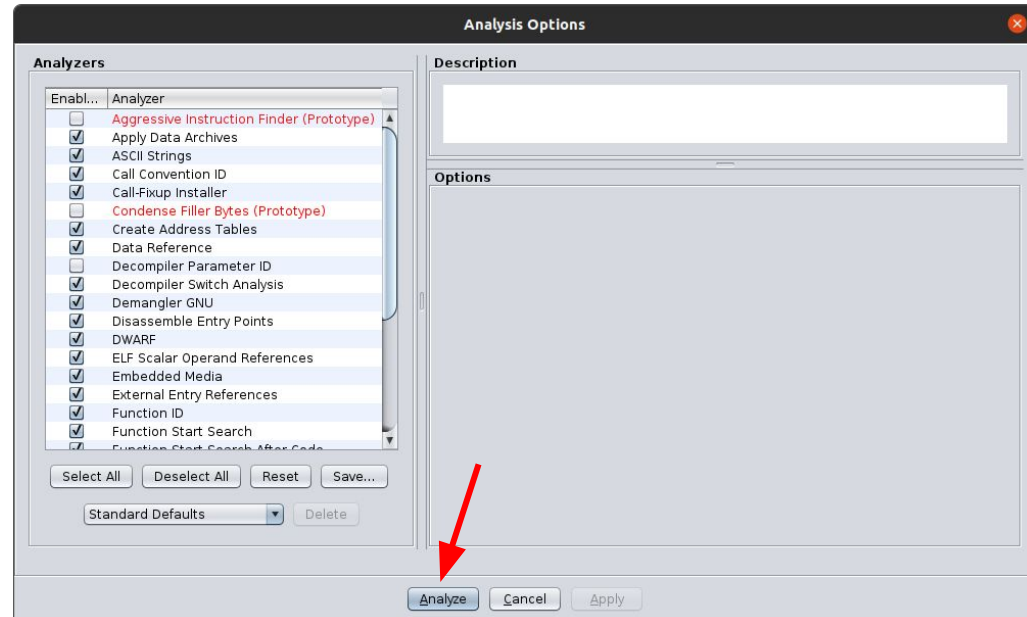
Esercizio guidato

- Ghidra chiederà se analizzare il file, rispondiamo “Sì”



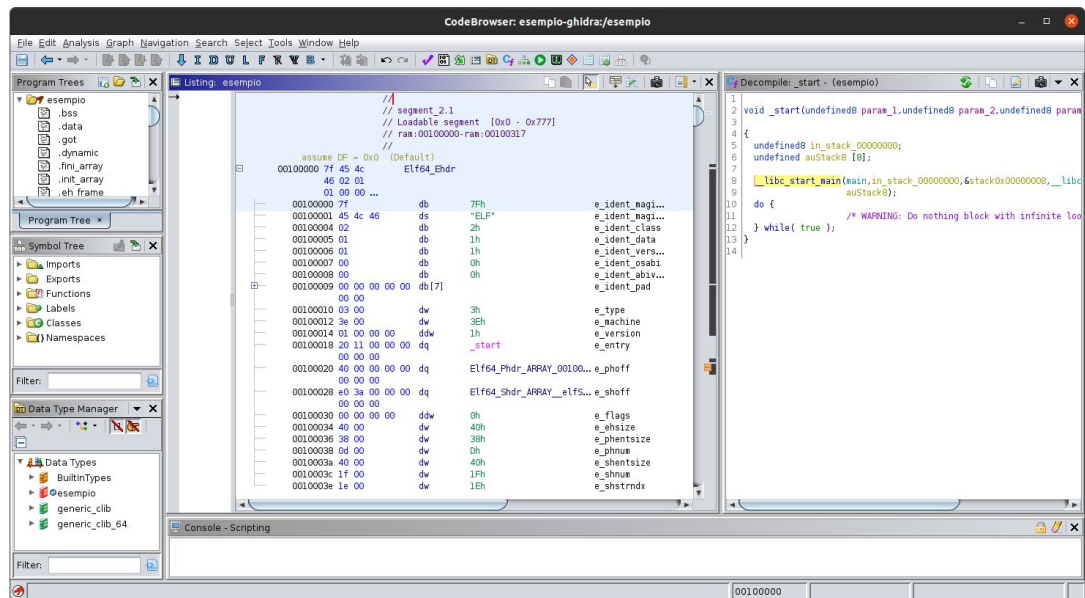
Esercizio guidato

- Possiamo lasciare le impostazioni predefinite e cliccare su “Analyze”



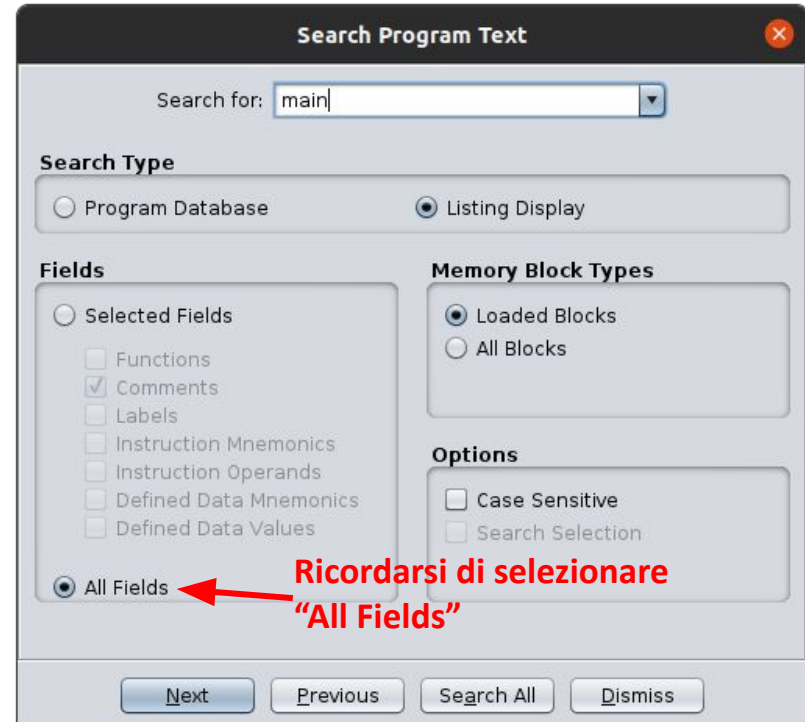
Esercizio guidato

- A questo punto Ghidra avrà disassemblato il file e presenterà l'assembly sulla sinistra e il codice sulla destra



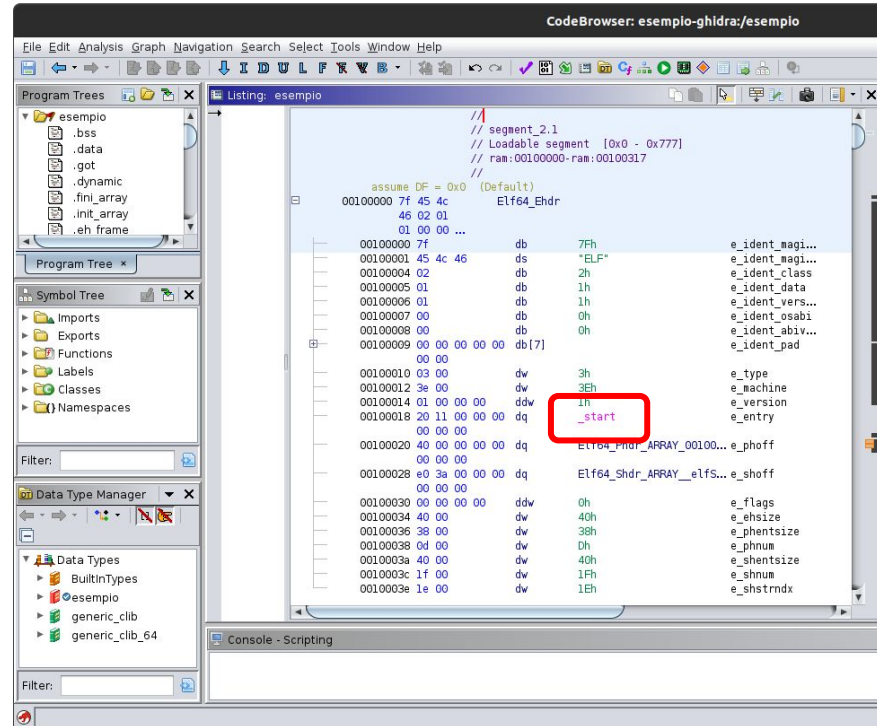
Esercizio guidato

- Spesso è utile iniziare la propria analisi partendo dal “main”. Per fare ciò si può cercare la parola “main” dentro al testo del programma (“Search” → “Program Text”)



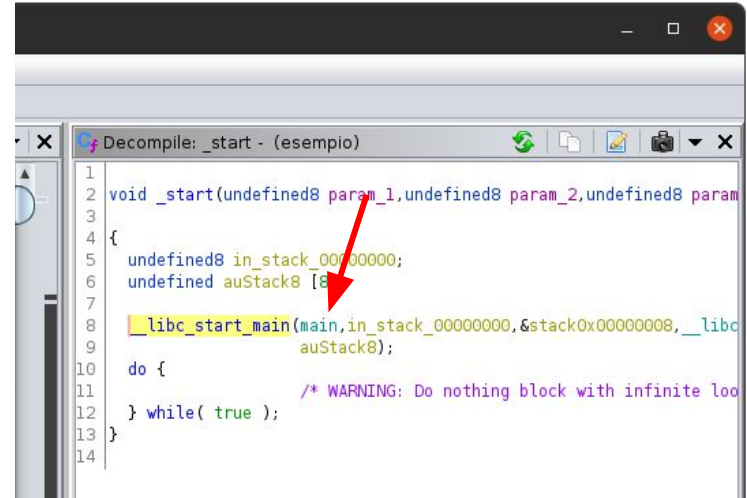
Esercizio guidato

- ...oppure si può fare doppio click sulla funzione **_start** o **entry** nell'assembly...



Esercizio guidato

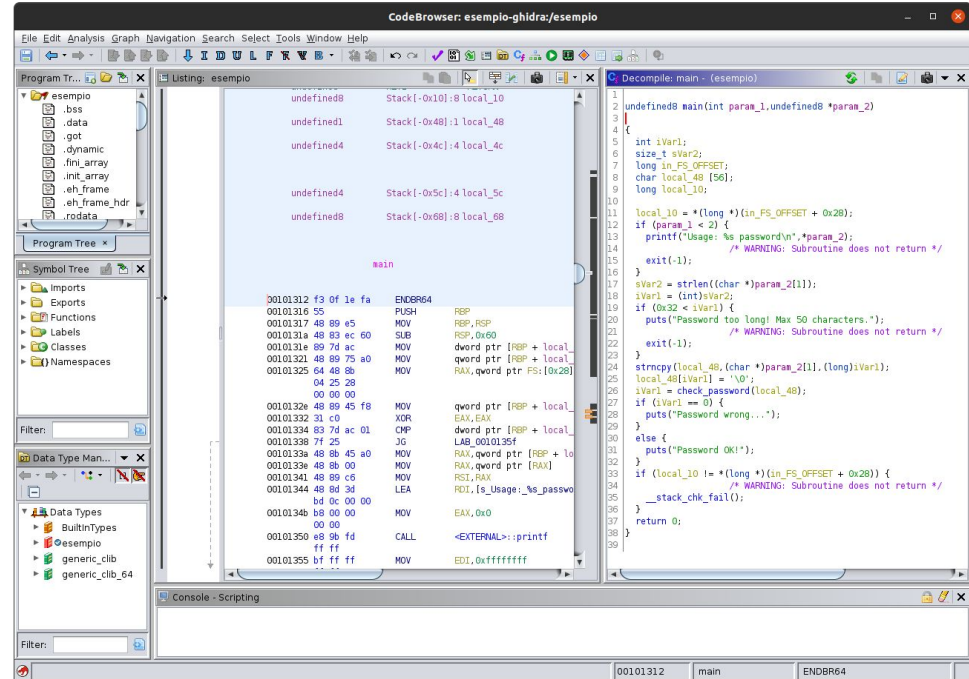
- ...che mostrerà il seguente codice (o simile), dove potremo trovare la funzione **main** come primo argomento (a volte riconosciuta come **FUN_<numeri>**) e farci doppio click sopra per entrarci



```
1 void _start(undefined8 param_1,undefined8 param_2,undefined8 param_3)
2 {
3     undefined8 in_stack_00000000;
4     undefined auStack8 [8];
5     libc_start_main(main,in_stack_00000000,&stack0x00000008,__libc_auStack8);
6     do {
7         /* WARNING: Do nothing block with infinite loop */
8     } while( true );
9 }
```

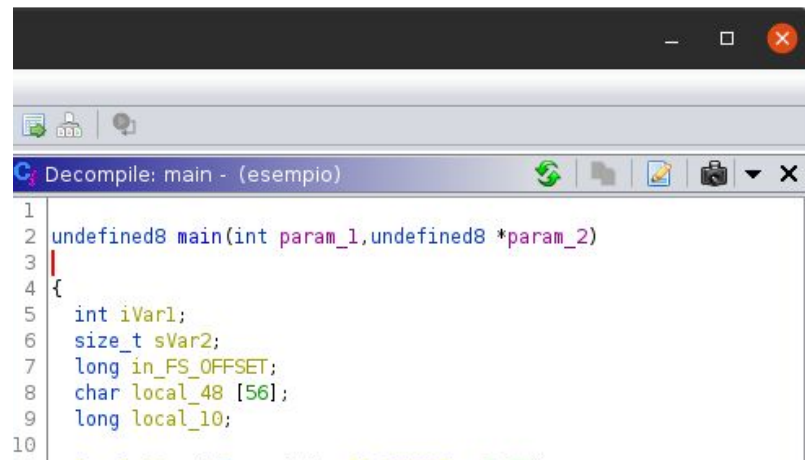
Esercizio guidato

- A questo punto possiamo vedere il codice del main



Esercizio guidato

- Per iniziare, notiamo che la *signature* della funzione main è strana: siamo abituati a vederla dichiarata come
`int main(int argc, char** argv)`
oppure come
`void main(int argc, char** argv)`

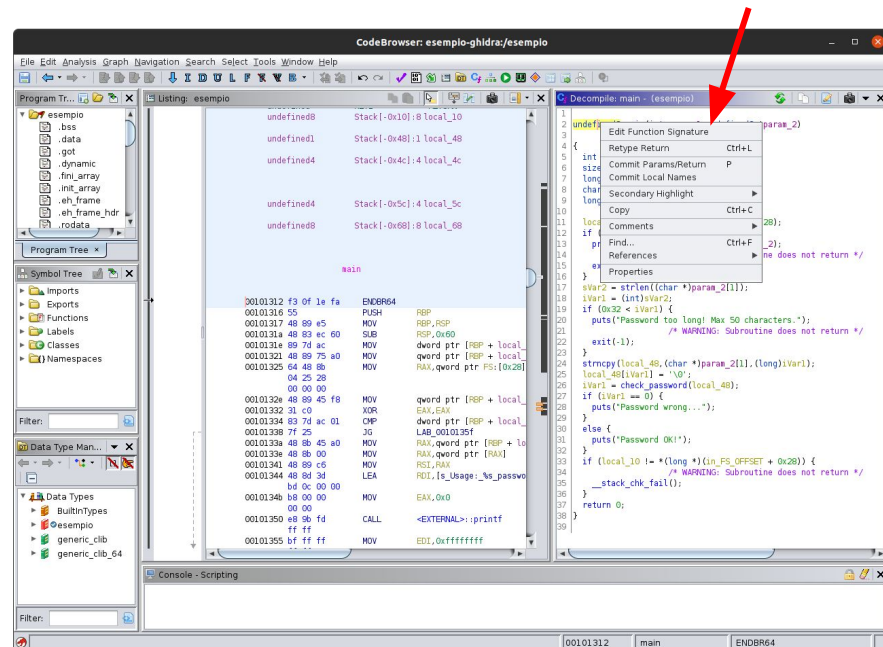
A screenshot of a decompiler window titled "Decompile: main - (esempio)". The window shows the decompiled C code for the main function. The code is as follows:

```
1
2 undefined8 main(int param_1,undefined8 *param_2)
3
4 {
5     int iVar1;
6     size_t sVar2;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
```

The code is displayed with line numbers on the left and standard C++ syntax highlighting. The window has a standard Windows-style title bar and a toolbar with icons for undo, redo, and other editing functions.

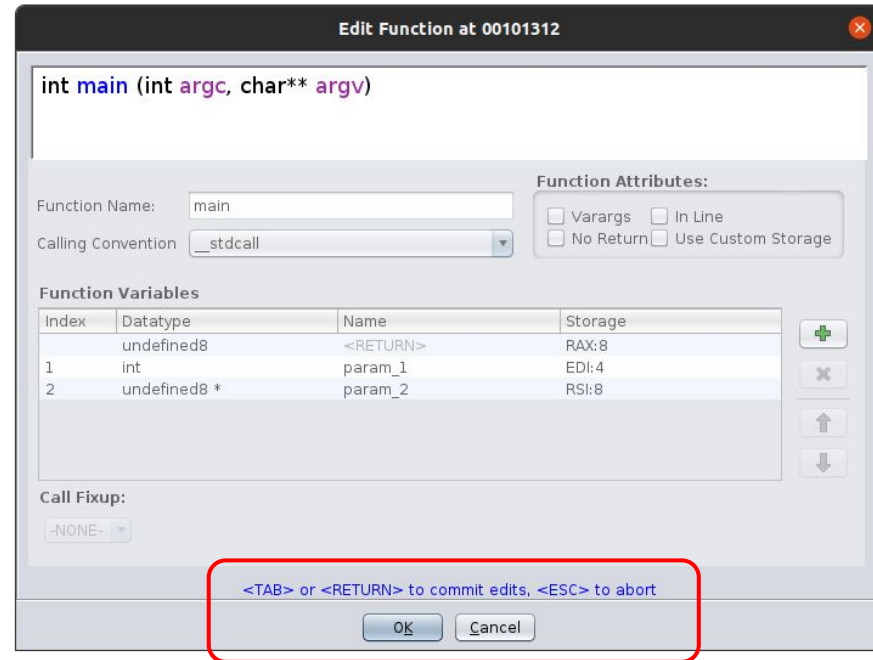
Esercizio guidato

- Quindi modifichiamo la signature della funzione in modo che rispetti il formato che conosciamo
- Facciamo click destro sulla funzione e clicchiamo su “Edit Function Signature”



Esercizio guidato

- Dopo aver scritto la signature correttamente bisogna premere INVIO (o TAB) e poi cliccare su OK



Esercizio guidato

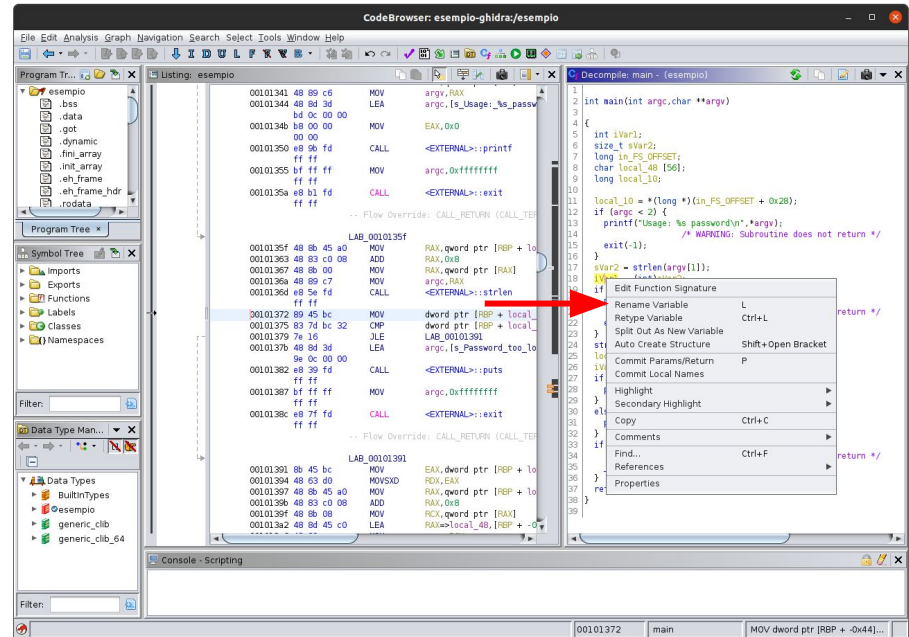
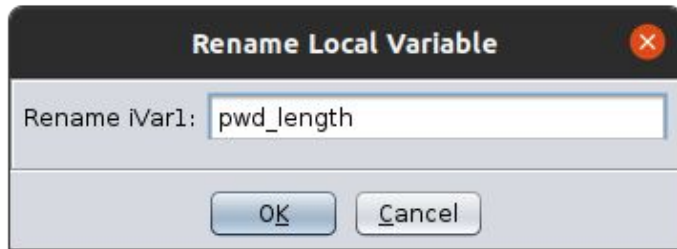
- Analizzando il codice potremmo voler rinominare alcune variabili per rendere più leggibile il codice
- Ad esempio, notiamo come, in questo caso, “iVar1” rappresenti la lunghezza della password inserita dall’utente



```
Decompile: main - (esempio)
1
2 undefined8 main(int param_1,undefined8 *param_2)
3 {
4
5     int iVar1;
6     size_t sVar2;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     if (param_1 < 2) {
13         printf("Usage: %s password\n",*param_2);
14         /* WARNING: Subroutine does not return */
15         exit(-1);
16     }
17     sVar2 = strlen((char *)param_2[1]);
18     iVar1 = (int)sVar2;
19     if (0x32 < iVar1) {
20         puts("Password too long! Max 50 characters.");
21         /* WARNING: Subroutine does not return */
22         exit(-1);
23     }
24     strcpy(local_48,(char *)param_2[1],(long)iVar1);
25     local_48[iVar1] = '\0';
26     iVar1 = check_password(local_48);
27     if (iVar1 == 0) {
28         puts("Password wrong...");
29     }
30     else {
31         puts("Password OK!");
32     }
33     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
34         /* WARNING: Subroutine does not return */
35         __stack_chk_fail();
36     }
37     return 0;
38 }
39
```

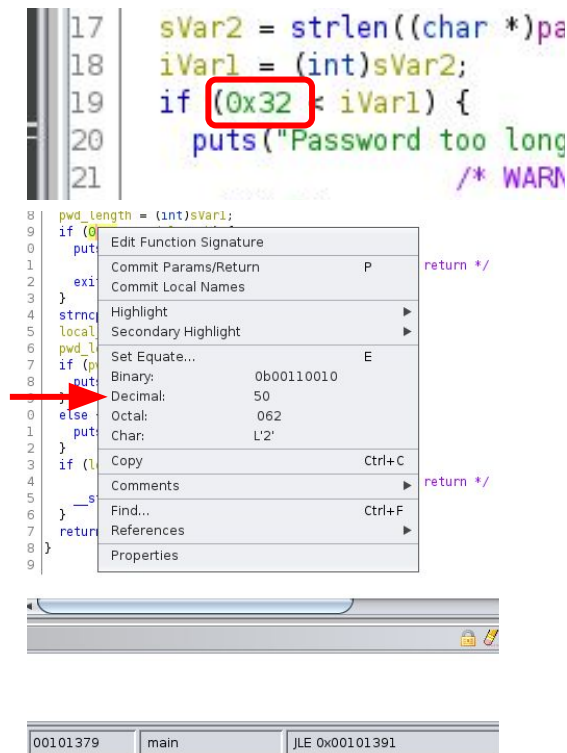
Esercizio guidato

- Per fare ciò clicchiamo su “Rename Variable” e diamo un nuovo nome alla variabile



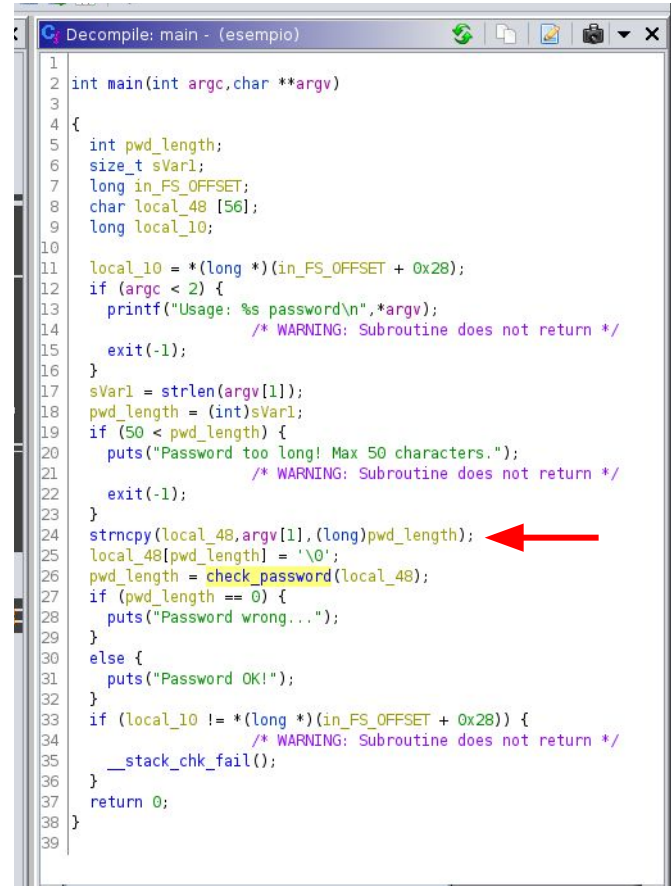
Esercizio guidato

- In Ghidra, di default, i numeri sono scritti in esadecimale
- Per cambiarne il formato, ad esempio per leggerli in decimale, facciamo click destro sul numero e poi clicchiamo su “Decimal”



Esercizio guidato

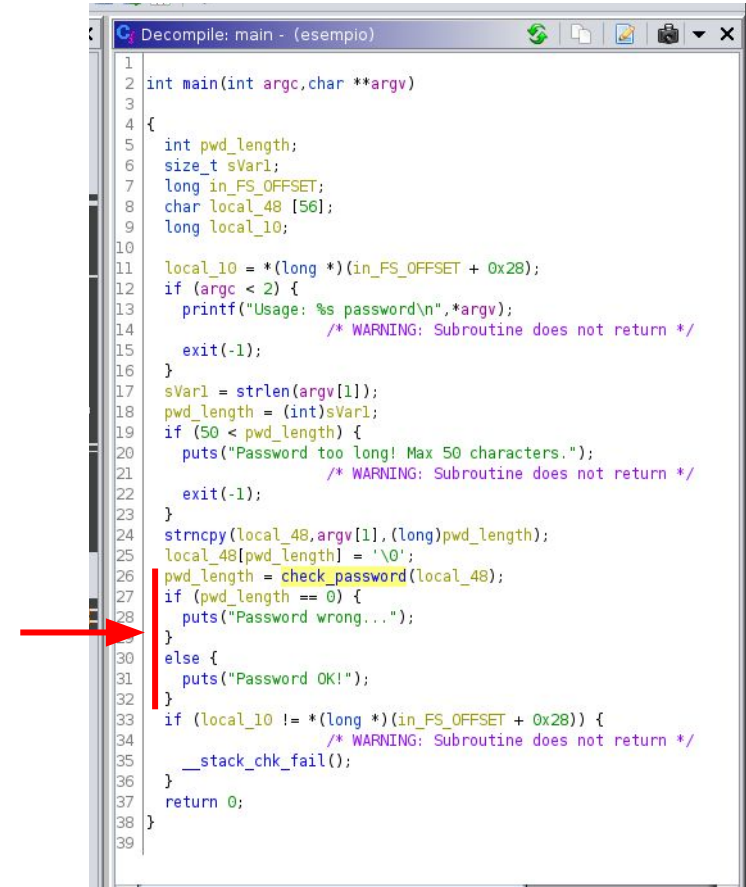
- Proseguendo con l'analisi notiamo che argv[1], la password inserita dall'utente come parametro a riga di comando, viene copiata dentro una variable “local_48”, che possiamo supporre sia un buffer



```
Decompile: main - (esempio)
1
2 int main(int argc, char **argv)
3
4 {
5     int pwd_length;
6     size_t sVar1;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     if (argc < 2) {
13         printf("Usage: %s password\n", *argv);
14         /* WARNING: Subroutine does not return */
15         exit(-1);
16     }
17     sVar1 = strlen(argv[1]);
18     pwd_length = (int)sVar1;
19     if (50 < pwd_length) {
20         puts("Password too long! Max 50 characters.");
21         /* WARNING: Subroutine does not return */
22         exit(-1);
23     }
24     strncpy(local_48, argv[1], (long)pwd_length);
25     local_48[pwd_length] = '\0';
26     pwd_length = check_password(local_48);
27     if (pwd_length == 0) {
28         puts("Password wrong...");
29     }
30     else {
31         puts("Password OK!");
32     }
33     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
34         /* WARNING: Subroutine does not return */
35         __stack_chk_fail();
36     }
37     return 0;
38 }
39
```

Esercizio guidato

- A questo punto si arriva alla funzione “check_password”
- Il suo risultato determina se il programma accetterà o meno la password
- Sembra che, per motivi di efficienza, il compilatore abbia deciso di riutilizzare la variabile “pwd_length” per memorizzare il valore di ritorno della funzione check_password

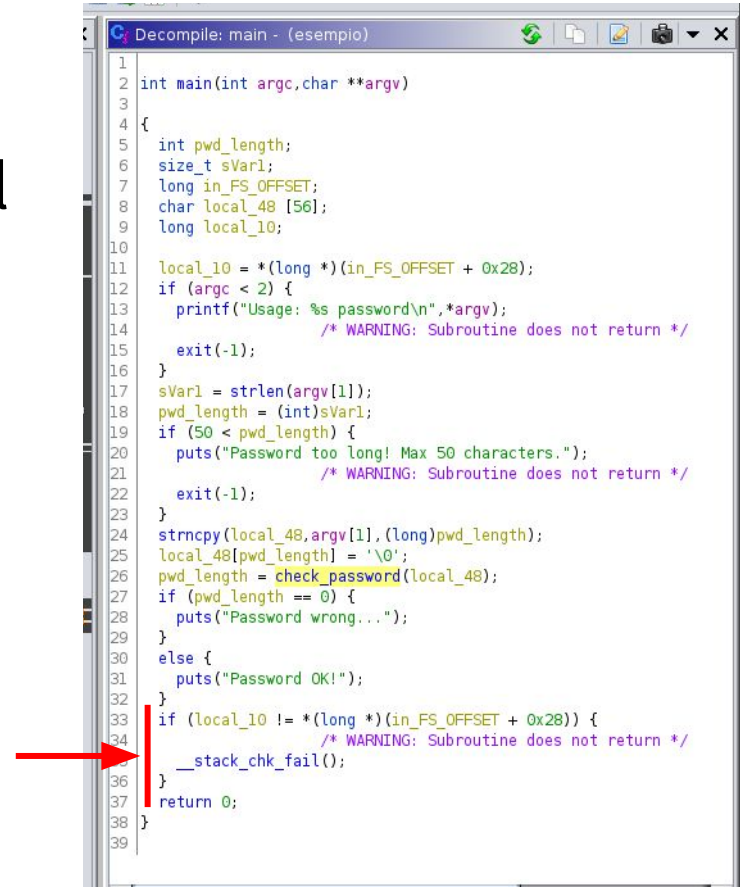


```
Decompile: main - (esempio)
1
2 int main(int argc, char **argv)
3
4 {
5     int pwd_length;
6     size_t sVar1;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     if (argc < 2) {
13         printf("Usage: %s password\n", *argv);
14         /* WARNING: Subroutine does not return */
15         exit(-1);
16     }
17     sVar1 = strlen(argv[1]);
18     pwd_length = (int) sVar1;
19     if (50 < pwd_length) {
20         puts("Password too long! Max 50 characters.");
21         /* WARNING: Subroutine does not return */
22         exit(-1);
23     }
24     strncpy(local_48, argv[1], (long) pwd_length);
25     local_48[pwd_length] = '\0';
26     pwd_length = check_password(local_48);
27     if (pwd_length == 0) {
28         puts("Password wrong...");
29     }
30     else {
31         puts("Password OK!");
32     }
33     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
34         /* WARNING: Subroutine does not return */
35         __stack_chk_fail();
36     }
37     return 0;
38 }
39
```

A red arrow points to line 26, where the result of the `check_password` function is stored in the `pwd_length` variable.

Esercizio guidato

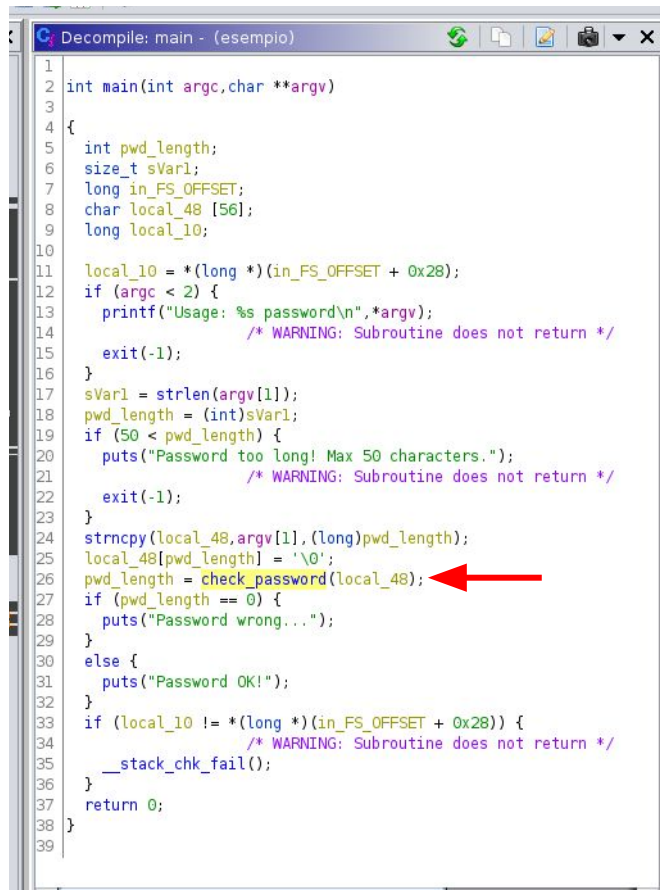
- Possiamo ignorare il resto del codice, in quanto verifica se si è in una situazione di stack overflow, e se sì, termina l'esecuzione del programma



```
Decompile: main - (esempio)
1
2 int main(int argc, char **argv)
3
4 {
5     int pwd_length;
6     size_t sVar1;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     if (argc < 2) {
13         printf("Usage: %s password\n", argv);
14         /* WARNING: Subroutine does not return */
15         exit(-1);
16     }
17     sVar1 = strlen(argv[1]);
18     pwd_length = (int) sVar1;
19     if (50 < pwd_length) {
20         puts("Password too long! Max 50 characters.");
21         /* WARNING: Subroutine does not return */
22         exit(-1);
23     }
24     strcpy(local_48, argv[1], (long) pwd_length);
25     local_48[pwd_length] = '\0';
26     pwd_length = check_password(local_48);
27     if (pwd_length == 0) {
28         puts("Password wrong...");
29     }
30     else {
31         puts("Password OK!");
32     }
33     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
34         /* WARNING: Subroutine does not return */
35         __stack_chk_fail();
36     }
37     return 0;
38 }
39
```


Esercizio guidato

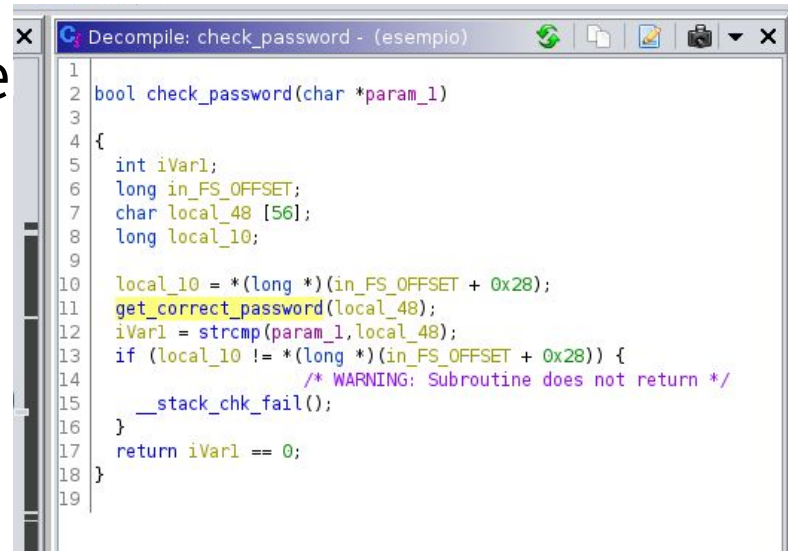
- Facciamo quindi doppio click su `check_password` per vederne il codice



```
Decompile: main - (esempio)
1
2 int main(int argc, char **argv)
3
4 {
5     int pwd_length;
6     size_t sVar1;
7     long in_FS_OFFSET;
8     char local_48 [56];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     if (argc < 2) {
13         printf("Usage: %s password\n", *argv);
14         /* WARNING: Subroutine does not return */
15         exit(-1);
16     }
17     sVar1 = strlen(argv[1]);
18     pwd_length = (int) sVar1;
19     if (50 < pwd_length) {
20         puts("Password too long! Max 50 characters.");
21         /* WARNING: Subroutine does not return */
22         exit(-1);
23     }
24     strncpy(local_48, argv[1], (long) pwd_length);
25     local_48[pwd_length] = '\0';
26     pwd_length = check_password(local_48);
27     if (pwd_length == 0) {
28         puts("Password wrong...");
29     }
30     else {
31         puts("Password OK!");
32     }
33     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
34         /* WARNING: Subroutine does not return */
35         __stack_chk_fail();
36     }
37     return 0;
38 }
39
```

Esercizio guidato

- La funzione confronta la password inserita dall'utente con una variabile "local_48", la quale sembra essere costruita dalla funzione get_correct_password
- Vediamo quindi il codice di get_correct_password facendoci doppio click sopra

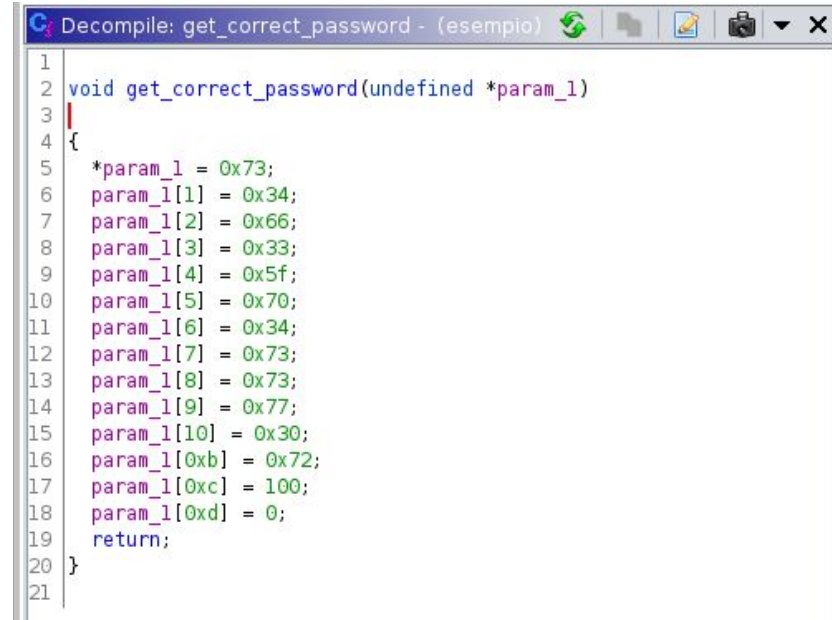


```
Decompile: check_password - (esempio)

1
2 bool check_password(char *param_1)
3
4 {
5     int iVar1;
6     long in_FS_OFFSET;
7     char local_48 [56];
8     long local_10;
9
10    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11    get_correct_password(local_48);
12    iVar1 = strcmp(param_1,local_48);
13    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
14        /* WARNING: Subroutine does not return */
15        __stack_chk_fail();
16    }
17    return iVar1 == 0;
18 }
19
```

Esercizio guidato

- Il comportamento della funzione sembra non essere chiarissimo, quindi proviamo a creare un programma che esegue questa funzione e vediamo che risultato dà



```
Decompile: get_correct_password - (esempio)

1
2 void get_correct_password(undefined *param_1)
3
4 {
5     *param_1 = 0x73;
6     param_1[1] = 0x34;
7     param_1[2] = 0x66;
8     param_1[3] = 0x33;
9     param_1[4] = 0x5f;
10    param_1[5] = 0x70;
11    param_1[6] = 0x34;
12    param_1[7] = 0x73;
13    param_1[8] = 0x73;
14    param_1[9] = 0x77;
15    param_1[10] = 0x30;
16    param_1[0xb] = 0x72;
17    param_1[0xc] = 100;
18    param_1[0xd] = 0;
19    return;
20 }
21
```

Esercizio guidato

- Effettuate le opportune modifiche...

```
soluzione.c > ...
1  #include <stdio.h>
2
3  void main()
4  {
5      char param_1[20];
6      param_1[0] = 0x73;
7      param_1[1] = 0x34;
8      param_1[2] = 0x66;
9      param_1[3] = 0x33;
10     param_1[4] = 0x5f;
11     param_1[5] = 0x70;
12     param_1[6] = 0x34;
13     param_1[7] = 0x73;
14     param_1[8] = 0x73;
15     param_1[9] = 0x77;
16     param_1[10] = 0x30;
17     param_1[0xb] = 0x72;
18     param_1[0xc] = 100;
19     param_1[0xd] = 0;
20     printf("%s\n", param_1);
21 }
22 |
```

Esercizio guidato

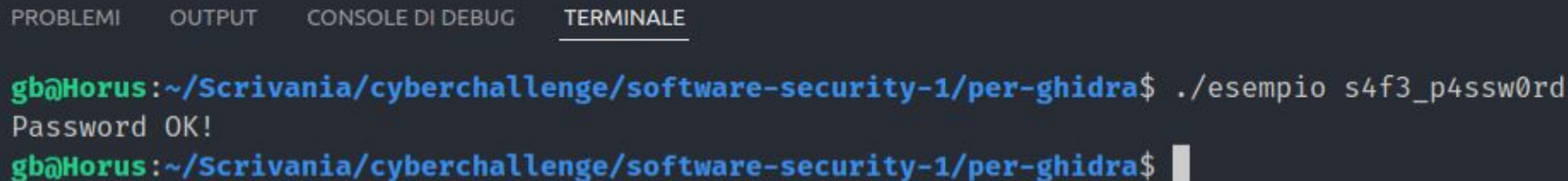
- ...possiamo compilare ed eseguire il programma.
Abbiamo ottenuto la password!

PROBLEMI OUTPUT CONSOLE DI DEBUG TERMINALE

```
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$ gcc soluzione.c -o soluzione
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$ ./soluzione
s4f3_p4ssw0rd
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$
```

Esercizio guidato

- Proviamo a fornirla al programma originale...



A screenshot of a terminal window with a dark background. At the top, there are four tabs: 'PROBLEMI', 'OUTPUT', 'CONSOLE DI DEBUG', and 'TERMINALE', with 'TERMINALE' being the active tab. The terminal shows a prompt 'gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra\$' followed by the command './esempio s4f3_p4ssw0rd'. The output is 'Password OK!'. The prompt is repeated on the next line with a cursor.

```
PROBLEMI OUTPUT CONSOLE DI DEBUG TERMINALE
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$ ./esempio s4f3_p4ssw0rd
Password OK!
gb@Horus:~/Scrivania/cyberchallenge/software-security-1/per-ghidra$
```

- Viene accettata. Abbiamo ottenuto la password corretta e l'esercizio è concluso!